BizPlus AI Requirements

Hackathon MVP and acceptance evidence

13 October 2026 • BizPlus AI Team • Three members • Six hours

We built a synthetic business transaction monitor that detects a missing customer outcome, shows the evidence, and verifies an approved recovery. This document gives the team and judges a concise product specification, architecture, acceptance evidence, and deployment checklist.

Problem and product outcome

A checkout response can look successful while the order confirmation never reaches the customer. The demo deliberately pauses a confirmation worker for one transaction. BizPlus AI catches the missing order despite HTTP 200, explains the supporting evidence, and resumes that worker only after approval. Verification checks the original order and the simulated payment count.

Audience and scope

Hackathon judges and the three-person team use the dashboard to explore one owned shop journey. All customer data and payments are synthetic. The prototype runs without an AI API key. Its visible diagnosis mode is Rules-based diagnosis.

Scenario	Required outcome
Healthy transaction	Login, inventory, cart, checkout, and confirmation pass their assertions.
Checkout unavailable	HTTP 500 occurs before a payment is created. Dependent confirmation is skipped.
Missing confirmation	Checkout succeeds. HTTP 200 with no order confirmation creates an incident.

User journey

The judge selects a scenario and presses Run Test. The dashboard shows each step progressing and records its actual result. The judge opens a failed incident, inspects its evidence, approves the supported recovery, and compares the original failure with verification. Auto-run can repeat a test every 30 seconds while the dashboard remains open.

The browser generates a demo-session ID to isolate records. Only one run may remain active in a session. Session IDs provide demo isolation rather than authenticated identity.

Functional requirements

Area	Required behavior
Execution	Make actual HTTP calls to fixed owned endpoints. Save status, expected and actual outcomes, assertion code, duration, timestamps, and relevant logs.
Investigation	Keep the failed step and evidence. Skip dependent steps and link the incident to its run.
Fingerprints	Group by workflow, step, endpoint, error category, and assertion code. Exclude timestamps and transaction IDs.
Diagnosis	Link rules to evidence. Separate observed facts from possible causes. Show no invented confidence percentage.
Recovery	Require explicit approval of resume_receipt_worker. Resume only the original order worker and verify its confirmation.
Idempotency	Preserve one payment and one confirmation through unique keys and transactional writes. Repeated recovery returns the existing verified result.
Metrics	Calculate history and recovery statistics from stored records. Keep original failed runs visible after recovery.

Architecture

React with Vite, TypeScript, Tailwind, and Recharts serves the dashboard. Express runs the API and HTTP workflow engine. Node 24 provides SQLite persistence. Frontend, backend, and shared workspaces live in one repository, and shared contracts compile before either application.

The runner calls a fixed sandbox hosted by the API. Evidence and incidents flow into SQLite. Rules create a stable fingerprint and suggested action. Approved recovery calls the sandbox worker and verifies the original order before updating incident state.

API and state contracts

Readiness is available at GET /health. GET /api/workflows lists the fixed workflow. POST /api/runs starts a scenario and GET /api/runs/:id returns its steps. Incident list and detail endpoints live at /api/incidents. POST /api/incidents/:id/recover accepts the supported approved action. GET /api/metrics returns calculated results.

Incident states are open, recovering, and resolved. Failed verification returns an incident to open with new evidence. Checkout HTTP 500 receives investigation guidance. The public runner never accepts arbitrary test URLs or command execution.

Acceptance evidence

Verification recorded 13 October 2026 10:27 IST. Scope: Local production backend and real HTTP sandbox replay. Durations below include the sandbox's deliberate 220 ms request delay so judges can follow the demo. They do not represent production workload performance.

Measured check	Result
Automated tests	11 of 11 passed
Healthy journey duration	1221 ms
Checkout 500 journey duration	1005 ms
Missing confirmation duration	1180 ms
Recovery verification duration	247 ms
Simulated payments before and after recovery	1 before and 1 after

Required test scenarios

Healthy runs must verify stock, matching transaction amount, and order identity. Checkout 500 must create no payment and skip confirmation. Missing confirmation must create a business-assertion incident even when HTTP status is 200. The diagnosis must reference evidence captured from that run.

Recovery must use the original order and leave its payment count unchanged. Repeated recovery must be harmless. Unsupported actions and incidents from another session must be rejected. A failed verification must remain open and retain its evidence. Dashboard metrics must match stored records.

Browser checks cover real API data, responsive layout, direct route refresh, backend loading and errors, and separate sessions. Release checks cover hosted cold start, incognito access, refresh after restart, a second device, and a tested local offline fallback.

Release verification status

Local API and browser journey verified. Public deployment and cold-start checks remain pending.

Completion criterion

A judge opens one public frontend URL and completes Run Test, Investigate, Approve Recovery, and Verified Result using actual sandbox evidence. Local verification alone does not establish completion of this public-hosting criterion.

Deployment and operational limits

Netlify builds from the repository root with npm run build:frontend and publishes frontend/dist. Set VITE_API_BASE_URL to the Render HTTPS origin before building. The SPA rewrite supports deep links. The browser bundle contains configuration only, never secrets.

Render builds at repository root with npm ci followed by npm run build:backend and starts with npm start. Pin Node 24.19.0. The API listens on 0.0.0.0 using PORT, exposes /health, initializes SQLite, and allows the exact Netlify production origin in CORS. The frontend disables runs until readiness and allows time for a sleeping service to wake.

Free hosted SQLite history can reset after sleep, restart, or deployment. The UI discloses temporary history and instance changes. Keep the tested local build and local SQLite ready for an offline presentation after installing dependencies.

Team ownership and checkpoints

Owner	Primary work
You	Backend, sandbox, runner, persistence, requirements and architecture.
Member 2	Dashboard, charts, responsive integration, screenshots and slides.
Member 3	Rules, fingerprints, recovery checks, deployment and demo rehearsal.

Agree contracts during the first 30 minutes. Start skeleton deployment by minute 60. Connect runs and diagnosis during hours one to three, finish recovery and get the core online by hour four, then freeze features. Use hour five for public verification and polish, and the final hour for artifacts and two rehearsals.

Existing systems and roadmap

New Relic already documents synthetic response assertions. Checkly documents AI root cause analysis. BizPlus AI does not claim an industry first. The hackathon demonstration combines a missing business outcome, readable evidence, approval, and original-order verification in one focused workflow.

Next work: authenticated operators, durable shared storage, and a recovery audit policy. Evaluate optional LLM summaries against evidence only after those foundations. Real payments, unrestricted workflow builders, and production services are outside this MVP.

References

Render monorepos: https://render.com/docs/monorepo-support

Render free service limits: https://render.com/docs/free

Netlify Vite setup: https://docs.netlify.com/build/frameworks/framework-setup-guides/vite/

New Relic synthetic assertions: https://docs.newrelic.com/docs/synthetics/synthetic-monitoring/scripting-monitors/write-synthetic-api-tests/

Checkly AI root cause analysis: https://www.checklyhq.com/docs/resolve/ai-root-cause-analysis/overview/